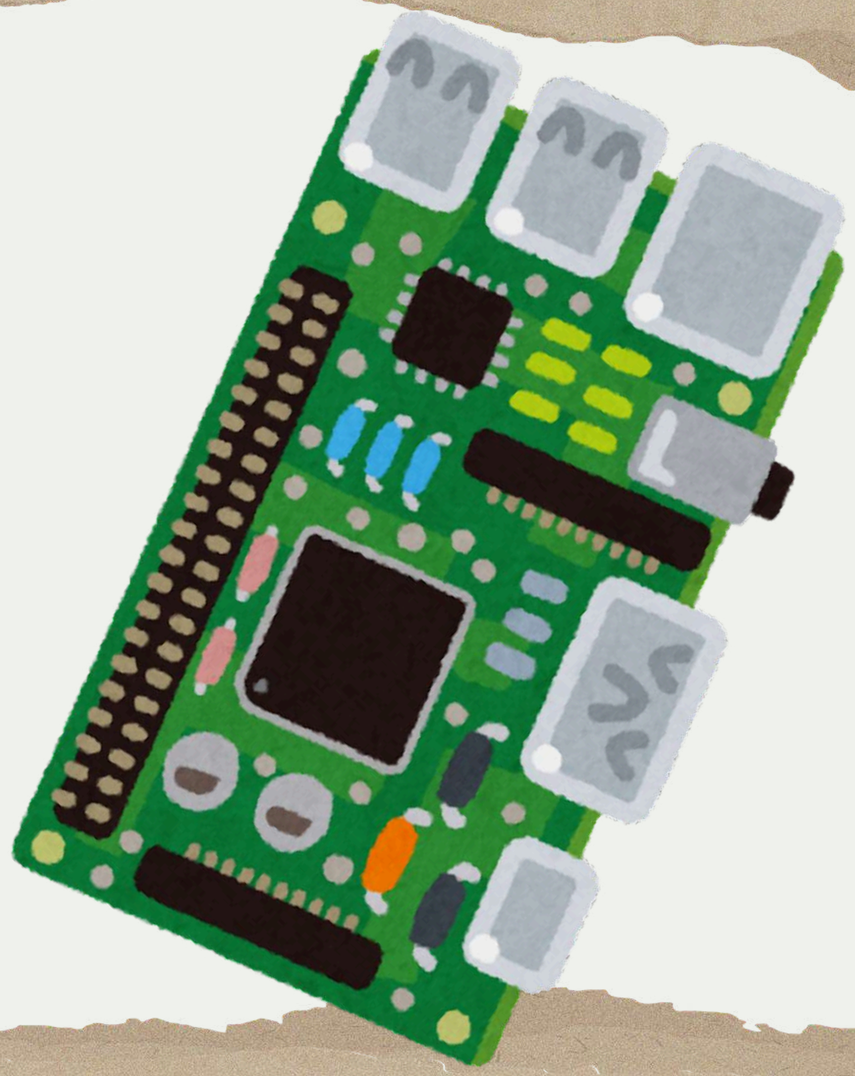


Access Control System

Team Members: **Salma Waleed, Salma Fouad, Yasmin Mohamed**

Team Name: **Bit.By.Bit**

Mentor: **GESTELL**

Course: **NTI**

Objectives

- Secure user authentication
- Password masking
- Failed-attempt counter
- Alarm activation
- Lockout mechanism
- Embedded software architecture
- Expandable design

WHAT IS THE OBJECTIVE?

What the project aims to achieve.

"Why are we building it?"
(Goals)

Features

- Password Authentication
- Admin Mode
- User Management
- Failed Attempt Counter
- Lockout Mode
- Alarm System
- LCD Interface
- Keypad Input
- EEPROM Storage

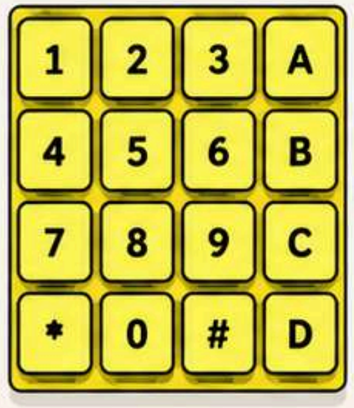
WHAT IS THE FEATURE?

What the project actually does.

"What capabilities does it have?"

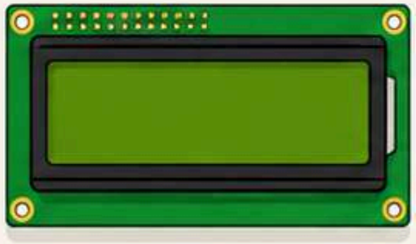
(Functions)

Hardware Components

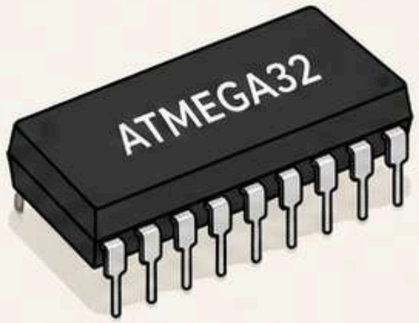


4×3 Keypad

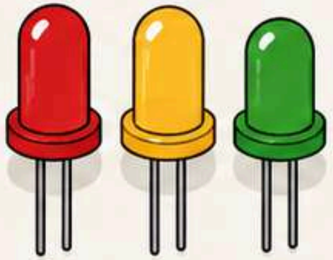
Buzzer



LCD 16×2



ATmega32

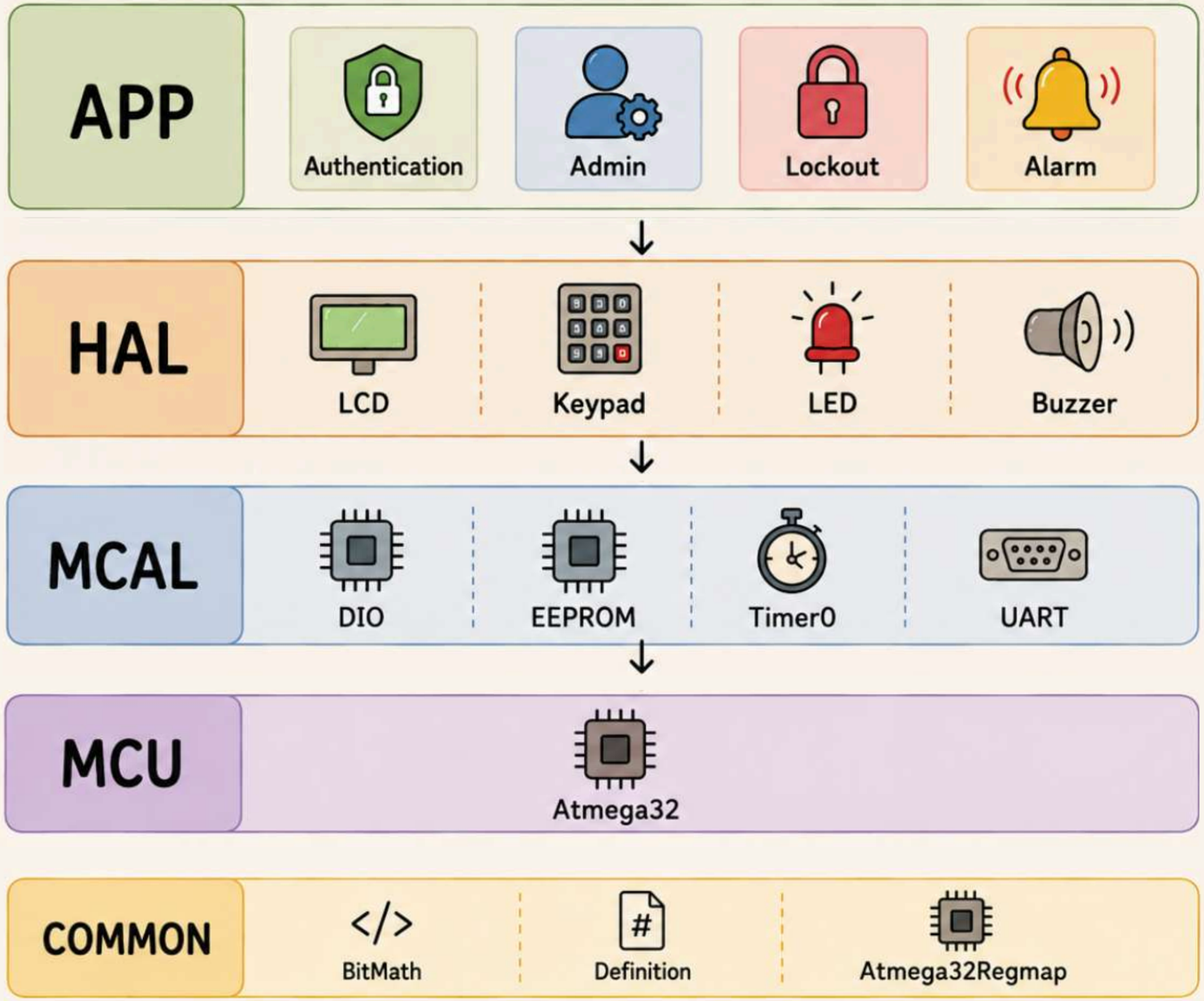


LEDs

EEPROM

Component	Purpose
ATmega32	Main Controller
4×3 Keypad	Password Entry
EEPROM	Stores User PINs
LCD 16×2	Displays Messages
Buzzer	Audible Alarm
LEDs	Status Indicators
UART	Debugging

Software Architecture



APP Modules

Authentication

- Receives password from keypad
- Compares with stored password
- Grants or denies access

Admin

- Add user
- Delete user
- Change password

Lockout

- Counts failed attempts
- Locks system after 3 failures

Alarm

- Activates buzzer
- Waits for alarm password
- Returns system to login

HAL Drivers

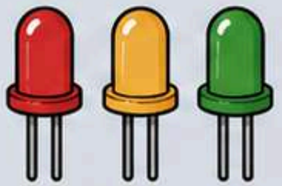
HAL (Hardware Abstraction Layer) provides drivers for hardware components used by the application.

LCD

Displays messages and menus to the user.
Supports 16x2 LCD.

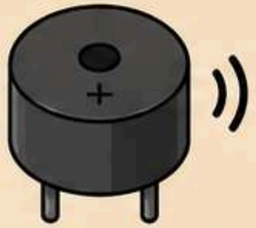
Keypad

Reads user input from 4x4 keypad.
Detects key press and returns the pressed key.



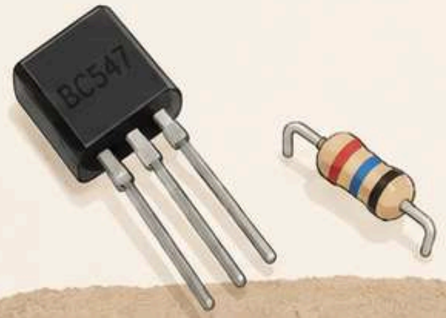
LED

Indicates system status using LED indicators.
Different patterns for different states.



Buzzer

Generates beep sound for alarm and alerts.
Used in security notification.



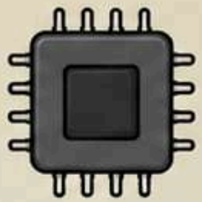
MCAL Drivers

MCAL (Microcontroller Abstraction Layer) provides low-level drivers that interact directly with the microcontroller peripherals.

```
// Set bit
#define SET_BIT(REG, BIT) \
  (REG |= (1 << BIT))

// Clear bit
#define CLR_BIT(REG, BIT) \
   (REG &= ~(1 << BIT))

// Read bit
#define GET_BIT(REG, BIT) \
  ((REG >> BIT) & 1)
```



DIO Driver

Controls digital I/O pins. Configures pin direction, reads input, and writes output.

EEPROM Driver

Reads from and writes to the external EEPROM. Stores users and passwords.

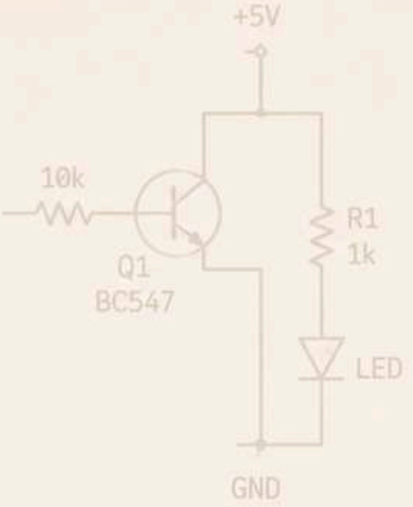
Timer0 Driver

Provides timing functions. Generates time delays and periodic events for the system.

UART Driver

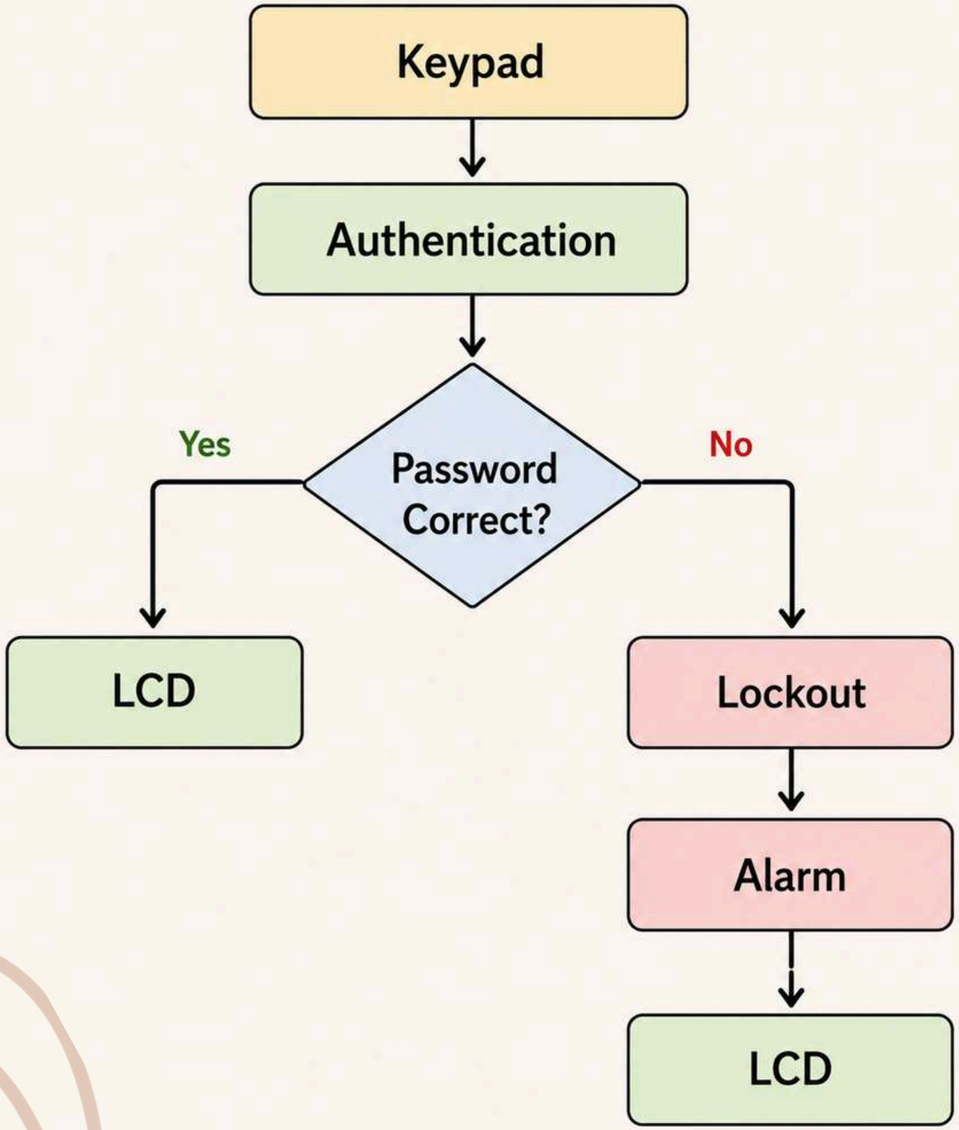
Handles serial communication. Used for debugging and data transfer.

TCCR0	0x53
TCNT0	0x52
OCR0	0x5C
TIMSK	0x59
TIFR	0x58




```
void delay_ms(uint16_t ms) {
  for(uint16_t i = 0; i < ms; i++) {
    TCNT0 = 0;
    while(!(TIFR & (1<<TOV0)));
    TIFR |= (1<<TOV0);
  }
}
```

Module Interaction



System Flow

Start

Initialize System

Enter Password

Correct?

Yes

Access Granted

No

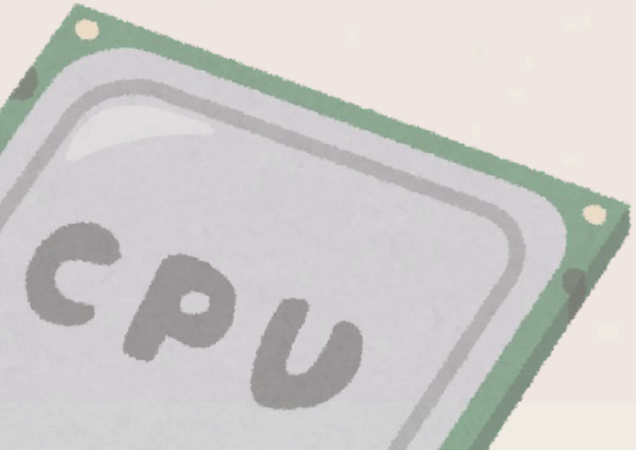
Increment Counter

No

Attempts = 3 ?

Yes

Lockout (Alarm ON)



Thank You

Q&A